第5章　Verilog HDL程序设计



5.1 组合电路设计

- 组合电路是指数字电路在任一时刻的输出仅取决于该时刻的输入。
- 简单门电路、编码器、解码器、多路开关、比较器、加法器、乘法器和校验电路等。

组合逻辑电路设计

- 组合逻辑电路设计可采用：
  - **assign语句；**
  - **always语句；**
  - 模块例化语句；
  - 以上语句混合方式

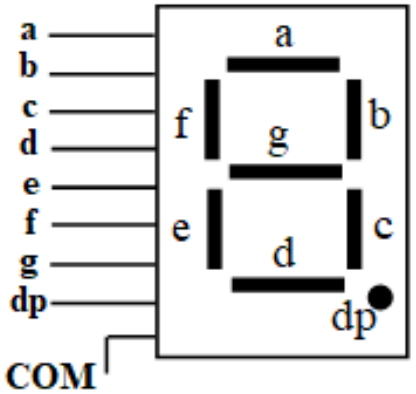
组合逻辑设计1：二输入异或门

```
module XOR_1 (A, B, Y);
    input A, B;
    output Y;
    assign Y = A ^ B;
endmodule

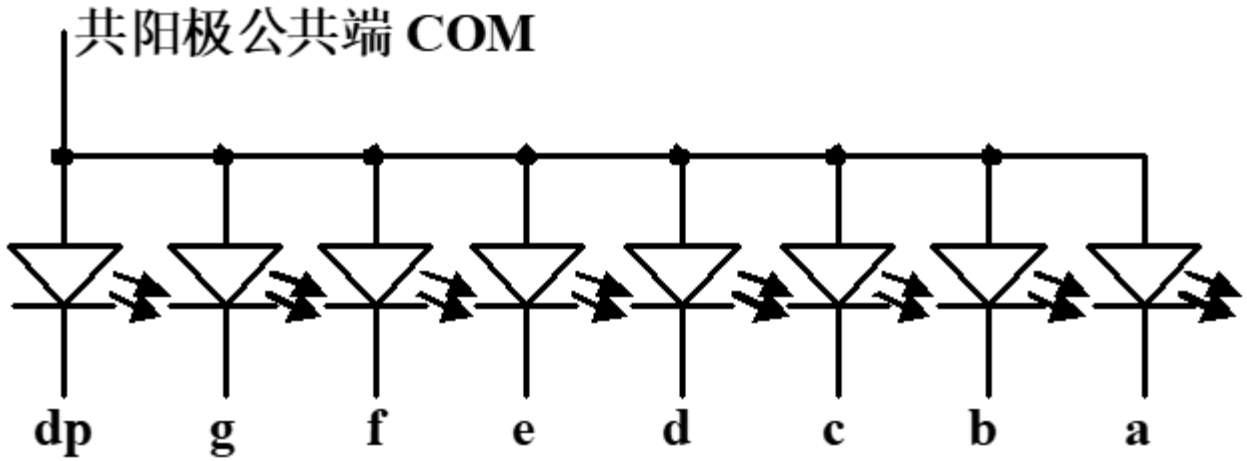
module XOR_3 (A, B, Y);
    input A, B;
    output Y;
    xor U1 (Y, A, B);
endmodule
```

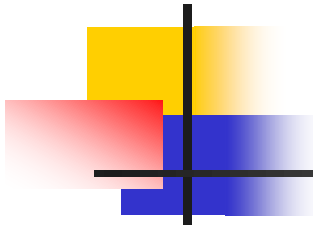
```
module XOR_2 (A, B, Y);
    input A, B;
    output Y;
    reg Y;
    always@(A, B)
      case ({A, B})
            2'b00: Y <= 0;
            2'b01: Y <= 1;
            2'b10: Y <= 1;
            2'b11: Y <= 0;
            Default: Y <= x;
      endcase
endmodule
```

组合逻辑设计2：七段数码管译码器



每个灯管亮的条件:
(1)共阳极端接高电平
(2)阴极端接低电平





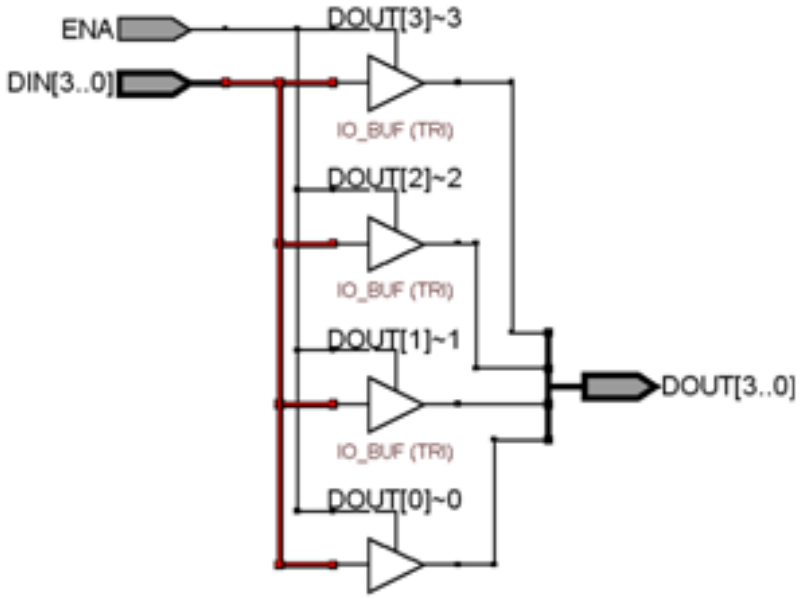
```
module Seg7Led (Din, A, B, C, D, E, F, G, DP, COM);
input  [3:0]  Din;
output  A, B, C, D, E, F, G, DP, COM;
reg  [6:0]  ATOG;
always   @  (Din)
begin
   ATOG = 7'b0000000;
   case (Din)
          4'b0000: ATOG = 7'b0000001;
          4'b0001: ATOG = 7'b1001111;
          4'b0010: ATOG = 7'b0010010;
          4'b0011: ATOG = 7'b0000110;
          4'b0100: ATOG = 7'b1001100;
          4'b0101: ATOG = 7'b0100100;
          4'b0110: ATOG = 7'b0100000;
          4'b0111: ATOG = 7'b0001111;
          4'b1000: ATOG = 7'b0000000;
          4'b1001: ATOG = 7'b0000100;
          4'b1010: ATOG = 7'b0001000;
          4'b1011: ATOG = 7'b1100000;
          4'b1100: ATOG = 7'b0110001;
          4'b1101: ATOG = 7'b1000010;
          4'b1110: ATOG = 7'b0110000;
          4'b1111: ATOG = 7'b0111000;
          Default: ATOG = 7'b0000000;
      endcase
end
assign A = ATOG[6];
assign B = ATOG[5];
assign C = ATOG[4];
assign D = ATOG[3];
assign E = ATOG[2];
assign F = ATOG[1];
assign G = ATOG[0];
assign DP = 1'b0;
assign COM = 1'b1;
endmodule
```

【例 5-30】8-3 编码器的 Verilog 程序

```
module Coder83 (D, Q);
  input [7:0] D;
  output [2:0] Q;
  reg [2:0] Q;
  always@(D)
      case (D)
          8'b00000001: Q <= 3'b000;
          8'b00000010: Q <= 3'b001;
          8'b00000100: Q <= 3'b010;
          8'b00001000: Q <= 3'b011;
          8'b00010000: Q <= 3'b100;
          8'b00100000: Q <= 3'b101;
          8'b01000000: Q <= 3'b110;
          8'b10000000: Q <= 3'b111;
          Default: Q <= 3'bxxx;
      endcase
endmodule
```

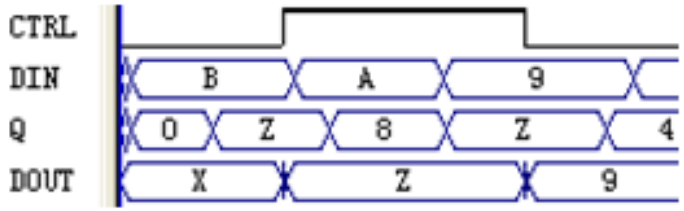
三态控制电路设计

```
module tri4B (ENA,DIN,DOUT);
   input ENA;
   input [3:0] DIN ;
   output [3:0] DOUT ;
   reg [3:0] DOUT;
   always @(DIN,ENA)
     if (ENA)  DOUT <= DIN ;
         else  DOUT <= 4'HZ;
endmodule
```

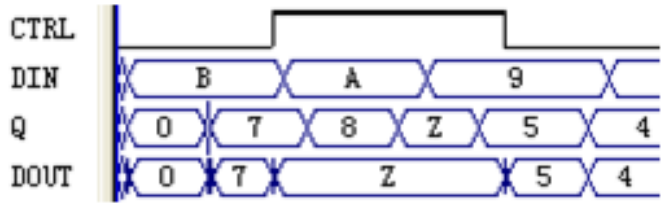


双向端口设计

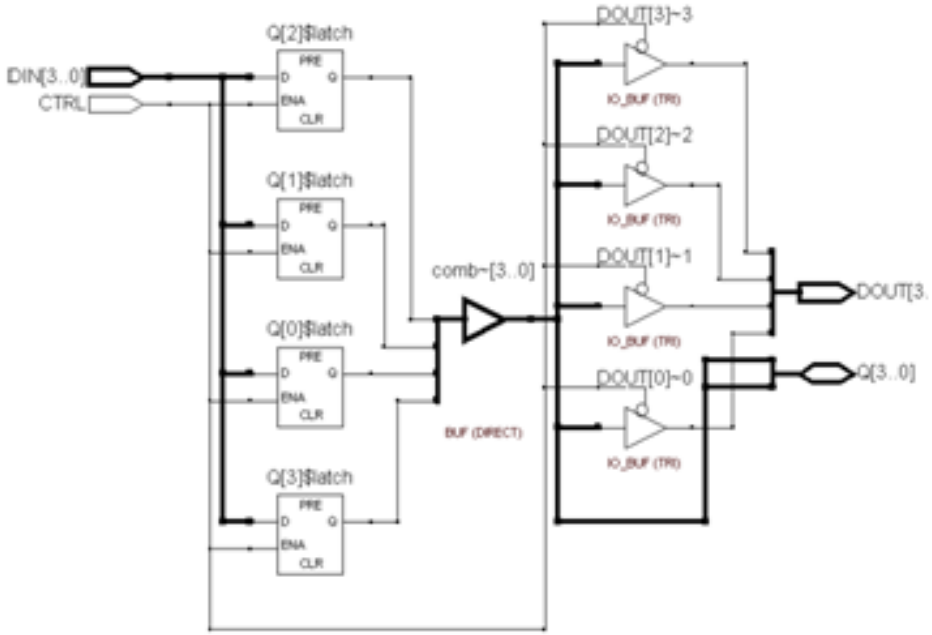
```
module BI4B(CTRL,DIN,Q,DOUT);
  input CTRL;     input[3:0] DIN;
  inout[3:0]Q; output[3:0] DOUT;
  reg [3:0] DOUT,Q ;
  always @(Q,DIN,CTRL)
     if (!CTRL)  DOUT<=Q ;
       else
  begin Q<=DIN; DOUT<=4'HZ;  end
    endmodule
```

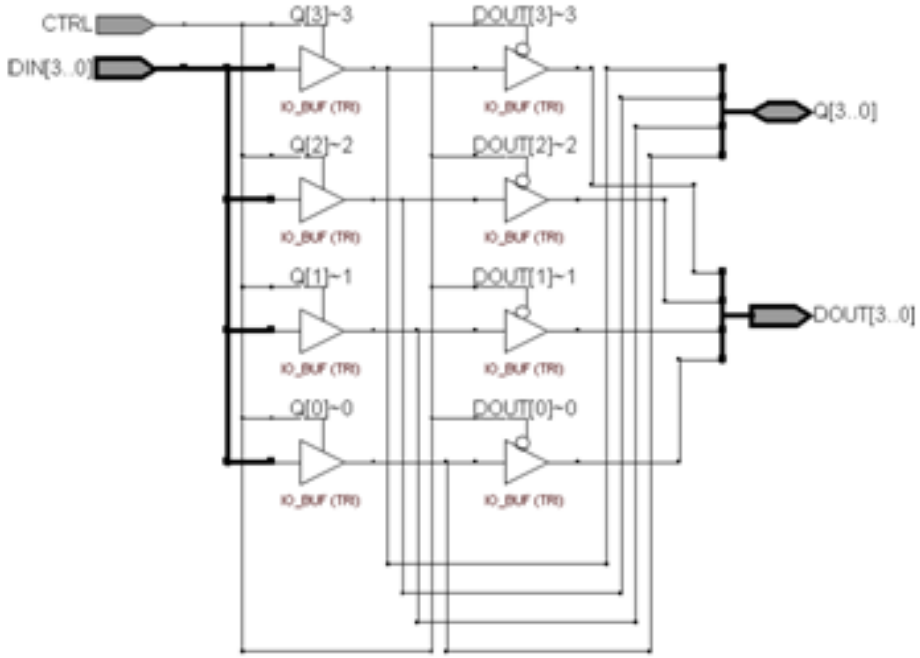



```
module BI4B(CTRL,DIN,Q,DOUT);
  input CTRL;     input[3:0] DIN;
  inout[3:0] Q; output[3:0] DOUT;
  reg [3:0] DOUT,Q ;
 always @(Q,DIN,CTRL)
 if (!CTRL)  begin DOUT<=Q;
       Q<=4'HZ; end  else
 begin Q<=DIN;  DOUT<=4'HZ;  end
 endmodule
```

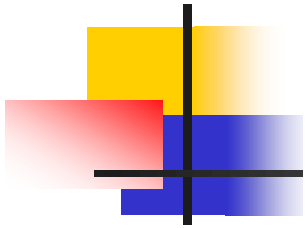


双向端口设计


DIN[3..0]
CTRL
Q[2]$latch
Q[1]$latch
Q[0]$latch
Q[3]$latch
PRE
D
Q
ENA
CLR
comb~[3..0]
BUF (DIRECT)
DOUT[3]~3
DOUT[2]~2
DOUT[1]~1
DOUT[0]~0
IO_BUF (TRI)
DOUT[3..
Q[3..0]


CTRL
DIN[3..0]
Q[3]~3
Q[2]~2
Q[1]~1
Q[0]~0
DOUT[3]~3
DOUT[2]~2
DOUT[1]~1
DOUT[0]~0
IO_BUF (TRI)
Q[3..0]
DOUT[3..0]

三态总线控制电路设计

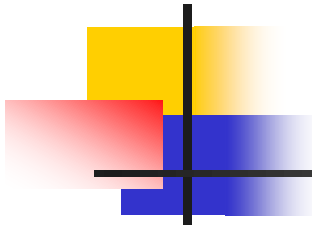


```
module triBUS4(
  IN3,IN2,IN1,IN0,ENA,DOUT);
  input[3:0] IN3,IN2,IN1,IN0 ;
  input[1:0]  ENA;
  output[3:0] DOUT;
  reg[3:0] DOUT;
  always @(ENA,IN3,IN2,IN1,IN0)
     begin
    if (ENA==0)  DOUT=IN3 ;
        else  DOUT=4'HZ;
    if (ENA==1)  DOUT=IN2 ;
        else  DOUT=4'HZ;
   if (ENA==2)  DOUT=IN1 ;
        else  DOUT=4'HZ;
   if (ENA==3)  DOUT=IN0 ;
        else  DOUT=4'HZ;
     end
endmodule
```

```
module triBUS4(
    IN3,IN2,IN1,IN0,ENA,DOUT);
  input[3:0] IN3,IN2,IN1,IN0 ;
  input[1:0]  ENA;
  output[3:0] DOUT; reg[3:0]DOUT;
  always @(ENA, IN0)
  if (ENA==2'b00)  DOUT=IN0;
     else  DOUT=4'hz;
  always @(ENA, IN1)
  if (ENA==2'b01)  DOUT=IN1;
     else  DOUT=4'hz;
  always @(ENA, IN2)
  if (ENA==2'b10)  DOUT=IN2;
     else  DOUT=4'hz;
  always @(ENA, IN3)
  if (ENA==2'b11)  DOUT=IN3;
     else  DOUT=4'hz;

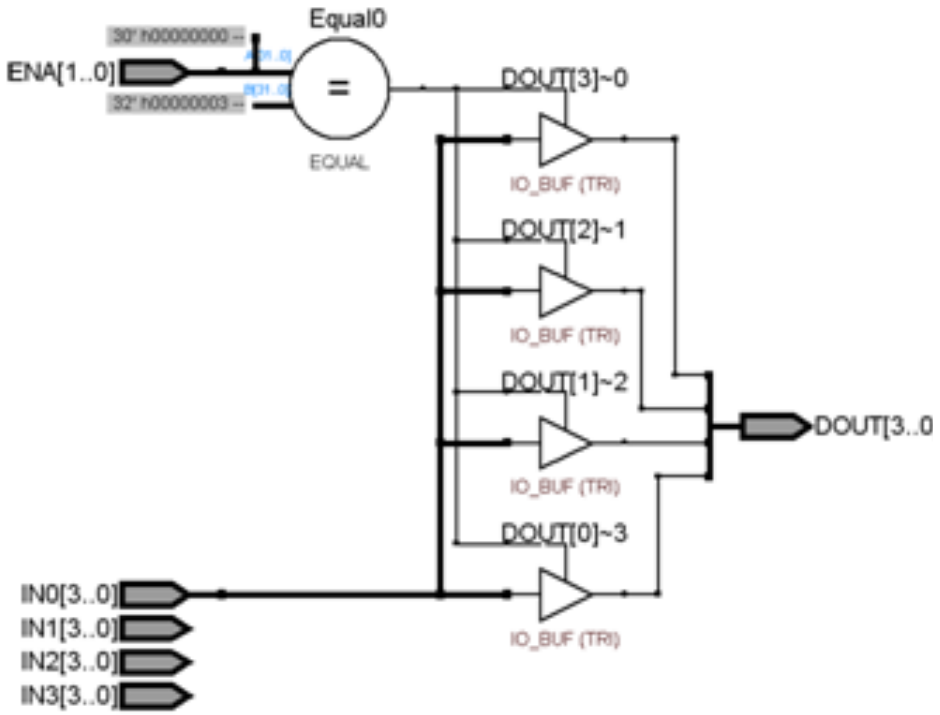
endmodule
```

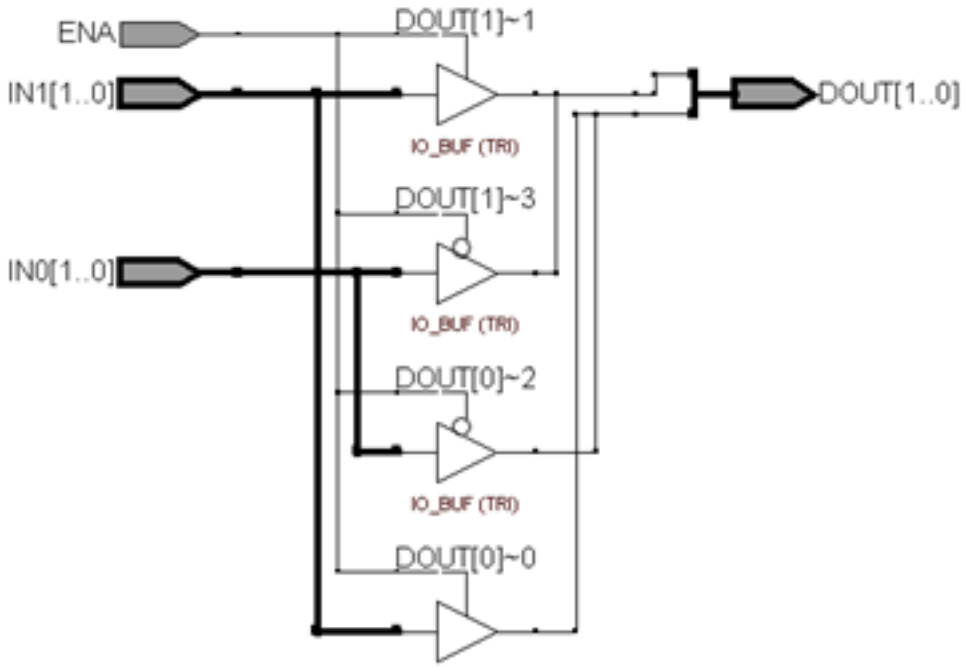
三态总线控制电路设计



```
module mux4_1(IN3,IN2,IN1,IN0,ENA,DOUT);
    input [3:0] IN3,IN2,IN1,IN0; input[1:0]  ENA;
     output[3:0] DOUT;
     assign DOUT = (ENA==2'B00) ? IN0 : 4'HZ;
     assign DOUT = (ENA==2'B01) ? IN1 : 4'HZ;
     assign DOUT = (ENA==2'B10) ? IN2 : 4'HZ;
     assign DOUT = (ENA==2'B11) ? IN3 : 4'HZ;
endmodule
```

三态总线控制电路设计


Equal0
ENA[1..0]
=
EQUAL
DOUT[3]~0
IO_BUF (TRI)
DOUT[2]~1
IO_BUF (TRI)
DOUT[1]~2
IO_BUF (TRI)
DOUT[0]~3
IO_BUF (TRI)
DOUT[3..0
IN0[3..0]
IN1[3..0]
IN2[3..0]
IN3[3..0]


ENA
IN1[1..0]
IN0[1..0]
DOUT[1]~1
IO_BUF (TRI)
DOUT[1]~3
IO_BUF (TRI)
DOUT[0]~2
IO_BUF (TRI)
DOUT[0]~0
DOUT[1..0]

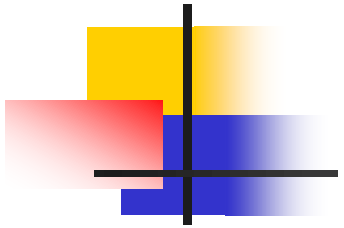
5.2 时序电路设计

- 时序电路是指数字电路在任一时刻的输出不仅取决于该时刻的输入，还取决于先前时刻的状态。
- 时序电路的驱动信号是时钟信号。
- 触发器、寄存器、计数器、分频器倍频器等。
- 时序电路设计用always语句（带时钟信号）。

程序实例（时序逻辑电路）

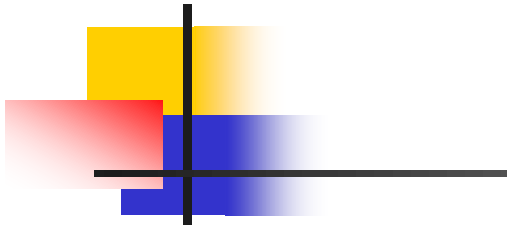
有异步复位的D触发器

```
module EDFF2(CLK, RST, D, Q);
   input CLK, RST, D;
   output  Q;
   reg Q;
   always @(posedge CLK or negedge RST)
         if (!RST) Q <= 0;
         else Q <= D;
endmodule
```



【例 5-10】 同步复位 D 触发器

```
module EDFF3(CLK, RST, D, Q);
  input CLK, RST, D;
  output  Q;
  reg Q;
  always @(posedge CLK)
      if (!RST) Q <= 0;
      else Q <= D;
endmodule
```



- 如果always语句的敏感信号表里采用posedge或negedge边沿触发，那么其他输入端口信号就不能列入敏感信号表了；
- 一旦CLK被定义为边沿敏感信号，它就不能在always结构中再出现了；
- always语句的敏感信号表中不允许有两个边沿触发信号；
- 表述为边沿敏感、实际为电平敏感的信号，如RST，只能作为异步复位或置位信号，不能作为一般的异步逻辑信号。

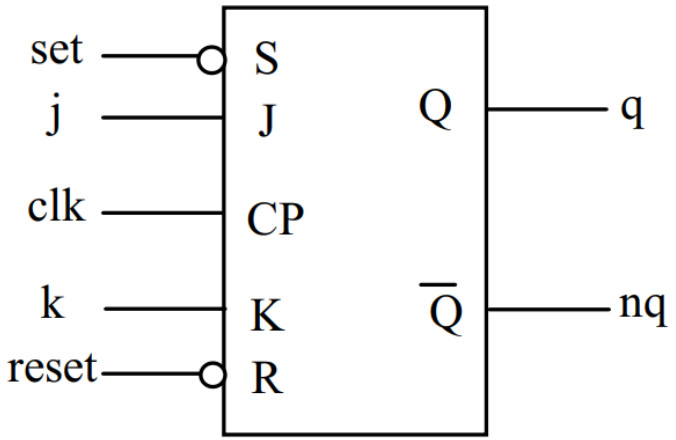


图 5-5 JK 触发器电路示意图

表 5-5 带异步置位/复位的 JK 触发器功能表

S	R	CP	J	K	Q	$\overline{Q}$
0	1	X	X	X	1	0
1	0	X	X	X	0	1
0	0	X	X	X	无效	
1	1	上升沿	0	0	保持	保持
1	1	上升沿	0	1	0	1
1	1	上升沿	1	0	1	0
1	1	上升沿	1	1	翻转	翻转
1	1	0	X	X	保持	保持

【例 5-11】JK 触发器

```
module myJKff (J, K, CLK, SET, RESET, Q, NQ);
  input J, K, CLK, SET, RESET;
  output  Q, NQ;
  reg Q, NQ;

  always @(posedge CLK or negedge SET or negedge RESET)
      if (!SET) begin Q <= 1; NQ <= 0; end
      else if (!RESET) begin Q <= 0; NQ <= 1; end
      else if (SET & RESET) begin
          if ((J) & (!K)) begin Q <= 1; NQ <= 0; end
          else if ((!J) & K) begin Q <= 0; NQ <= 1; end
          else if (J & K) begin Q <= ~Q; NQ <= ~NQ; end
          else begin Q <= Q; NQ <= NQ; end
      end
endmodule
```

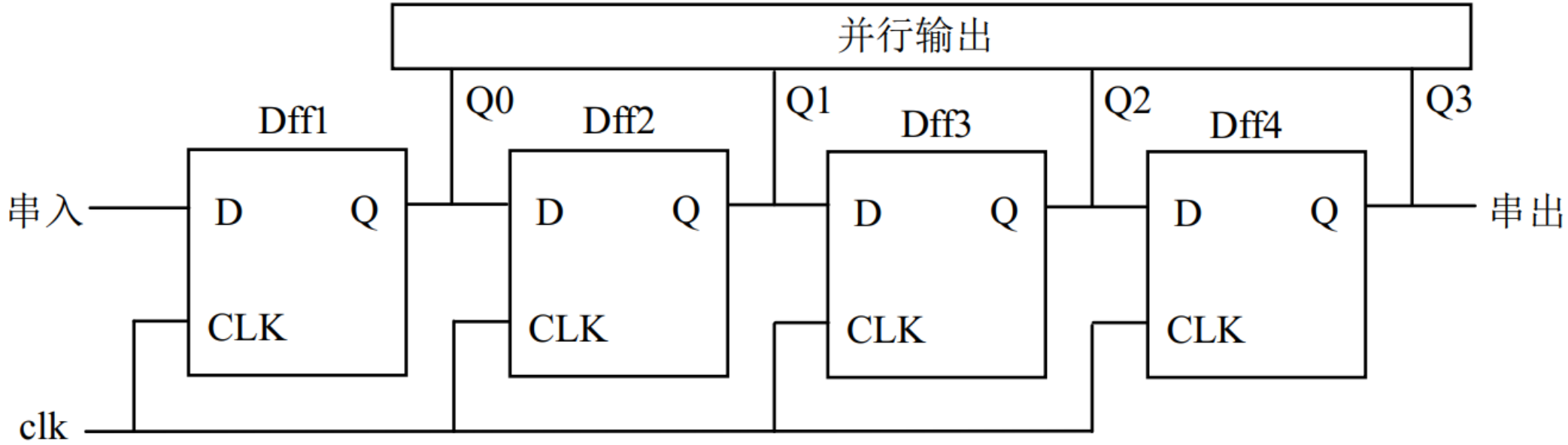


图 5-6　　D 触发器构成的串入/并出四位移位寄存器

【例 5-12】串入/并出四位移位寄存器

```
module ShiftReg1 (D, CLK, Q); //调用四个 D 触发器的串入/并出移位寄存器
  input D, CLK;
  output [3:0] Q;
  EDFF1 Dff1 (.CLK(CLK), .D(D), .Q(Q[0]));
  EDFF1 Dff2 (.CLK(CLK), .D(Q[0]), .Q(Q[1]));
  EDFF1 Dff3 (.CLK(CLK), .D(Q[1]), .Q(Q[2]));
  EDFF1 Dff4 (.CLK(CLK), .D(Q[2]), .Q(Q[3]));
endmodule

module ShiftReg2 (D, CLK, Q); //采用非阻塞赋值语句的串入/并出移位寄存器
  input D, CLK;
  output [3:0] Q;
  reg [3:0] Q;
  always @(posedge CLK)
    begin
      Q[0] <= D;
      Q[1] <= Q[0];
      Q[2] <= Q[1];
      Q[3] <= Q[2];
    end
endmodule
```

同步计数器是指在时钟脉冲的作用下，组成计数器的各个触发器的状态同时发生改变。例 5-14 是四位二进制同步计数器的 Verilog 实现程序，图 5-7 是它的 RTL 原理图，具有异步复位端、同步使能端和进位输出端。

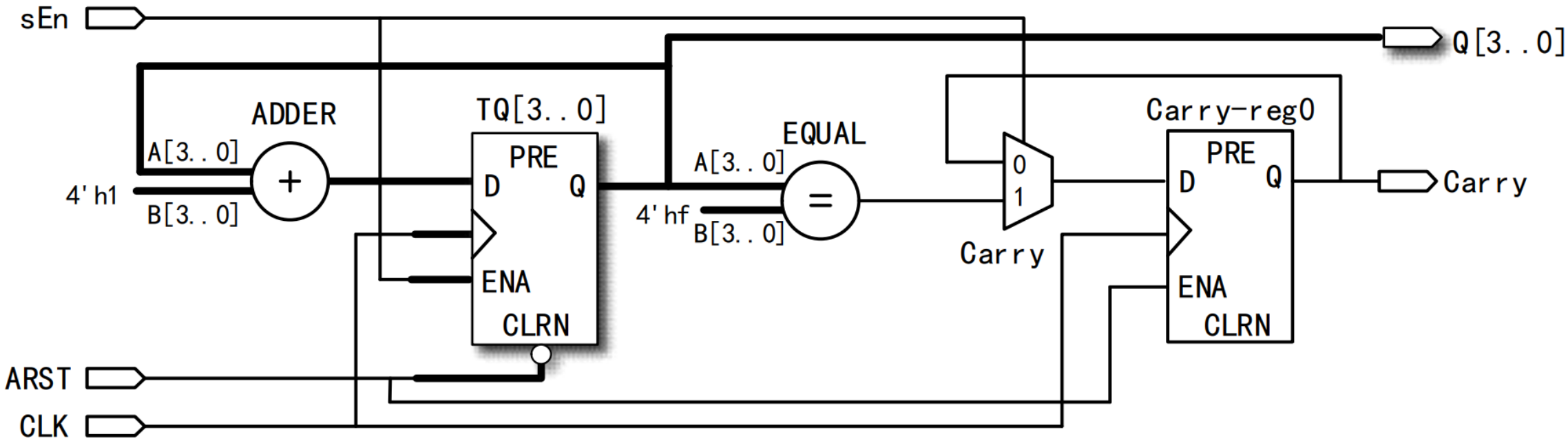


图 5-7 四位同步计数器的 RTL 原理图

【例 5-14】四位二进制同步计数器

```
module SyncCounter (CLK, ARST, sEn, Carry, Q);
  input CLK, ARST, sEn;
  output Carry;
  output [3:0] Q;
  reg Carry;
  reg [3:0] TQ;
      assign Q = TQ;
      always @(posedge CLK or negedge ARST)
      begin
          if (!ARST) TQ <= 4'b0000;
          else if (sEn) begin
              if (TQ==4'b1111)
                  begin
                  Carry <= 1'b1;
                  TQ <= 4'b0000;
                  end
              else
                  begin
                  Carry <= 1'b0;
                  TQ <= TQ + 1;
                  end
                  end
      end
endmodule
```

异步计数器是指构成计数器的各个触发器的状态不是在同一时钟的作用下同时改变，而是低位计数触发器的输出作为相邻高位计数触发器的时钟，通过低位触发器的输出来触发高位触发器。图 5-8 是利用四个 D 触发器构成的四位异步计数器，例 5-15 为其 Verilog 实现程序。在例 5-15 中，先设计了一个具有异步复位的 D 触发器，然后在四位异步计数器中将 D 触发器作为元件调用。在文件存放时，可以把 D 触发器单独存成一个文件，文件名为 AR_Dff.v，把四位异步计数器存为 AsyncCounter.v，其中.v 是 Verilog 源代码文件的后缀。

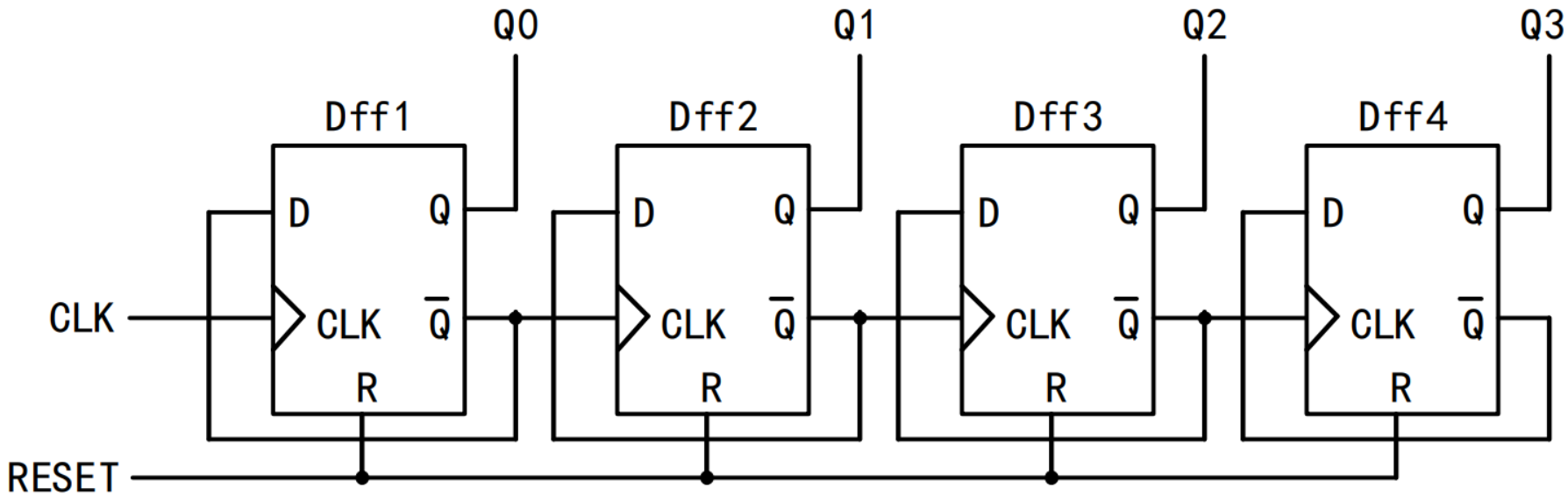


图 5-8 四位异步计数器电路示意图

【例 5-15】四位异步计数器

```
module AR_Dff (CLK, RESET, D, Q, NQ);// 异步复位 D 触发器
  input CLK, RESET, D;
  output  Q, NQ;
  reg Q, NQ;
  always @(posedge CLK or posedge RESET)
    if (RESET) begin Q <= 0; NQ <= 1; end
    else begin Q <= D; NQ <= ~D; end
endmodule

module AsyncCounter (CLK, RESET, Q);
  input CLK, RESET;
  output [3:0] Q;
  wire [1:4] TQ;
 AR_Dff Dff1 (.CLK(CLK),.RESET(RESET),.D(TQ[1]),.Q(Q[0]),.NQ(TQ[1]));
 AR_Dff Dff2 (.CLK(TQ[1]),.RESET(RESET),.D(TQ[2]),.Q(Q[1]),.NQ(TQ[2]));
 AR_Dff Dff3 (.CLK(TQ[2]),.RESET(RESET),.D(TQ[3]),.Q(Q[2]),.NQ(TQ[3]));
 AR_Dff Dff2 (.CLK(TQ[3]),.RESET(RESET),.D(TQ[4]),.Q(Q[3]),.NQ(TQ[4]));
endmodule
```

【例 5-16】2 的幂次方分频器

```
module Power2FD (clk, clk2D, clk4D, clk8D, clk16D);
  input clk;
  output clk2D, clk4D, clk8D, clk16D;
  reg [3:0] cnt;
  always @(posedge CLK)
    begin cnt <= cnt + 1; end
  assign clk2D = cnt[0];
  assign clk4D = cnt[1];
  assign clk8D = cnt[2];
  assign clk16D = cnt[3];
endmodule
```

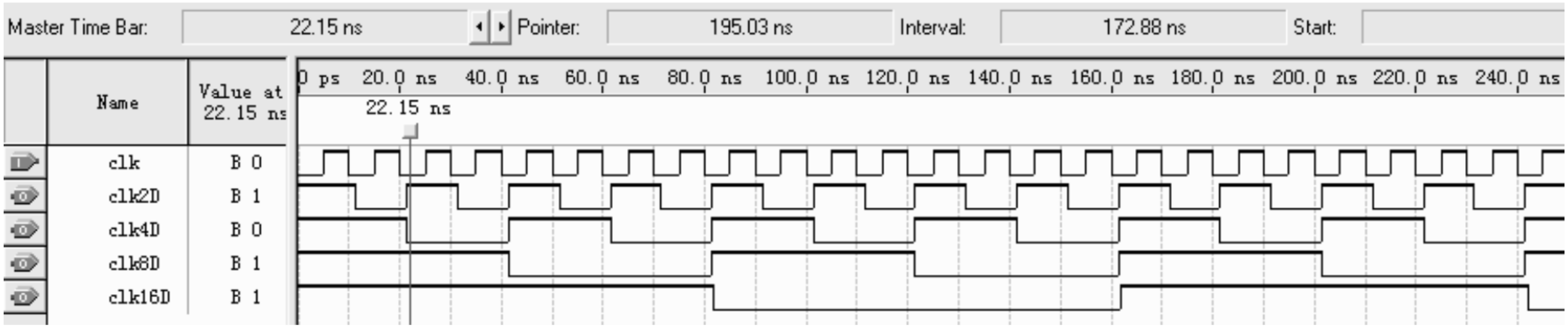


图 5-9　　例 5-16 的时序仿真波形

【例 5-17】偶数分频

```
module SixFD (clk, clk6D);
  input clk;
  output clk6D;
  reg clk6D;
  reg [1:0] cnt;
  always @(posedge clk)
    begin
    if (cnt==2'b10) begin
       cnt <= 2'b00;
       clk6D <= ~clk6D;
       end
    else cnt <= cnt + 1;
    end
endmodule
```

5.3 有限状态机

- 数字系统设计中常常会同时包含组合逻辑电路和时序逻辑电路，可以表示为：
- 时序逻辑的输入、输出分别叫做“次态”和“现态”；
- 状态数有限，形成有限状态机（Finite State Machine, FSM）模型。

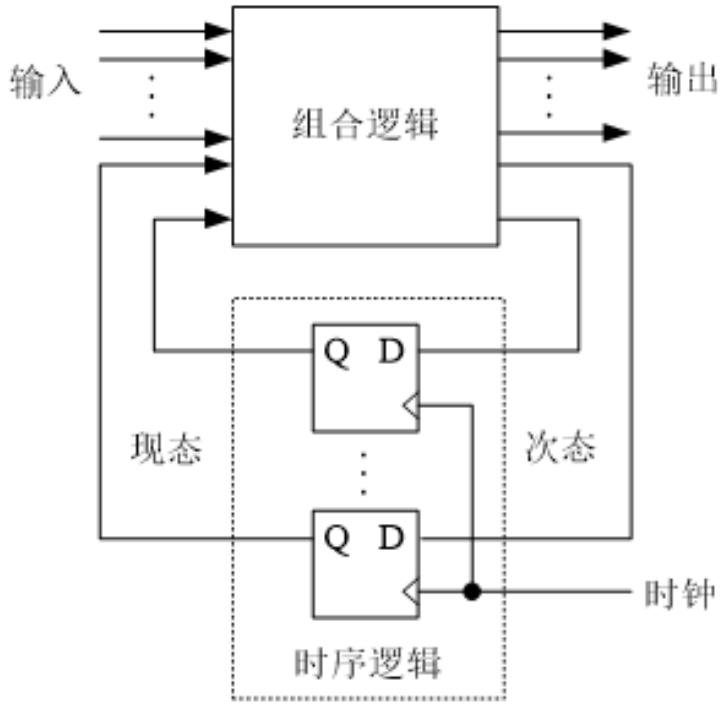


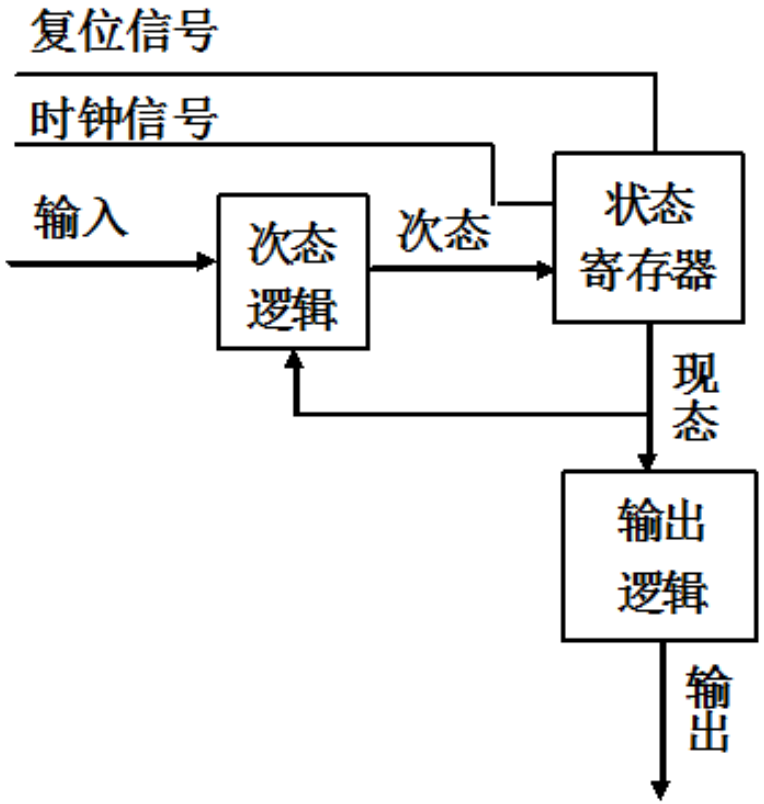
图 5-29 有限状态机模型

5.3 有限状态机

- 有限状态机（FSM）是数字系统设计中常用设计方法，任何时序模型都可以归结为状态机；
- 具有高效、高速、稳定、易于优化等优点但每个具体的电路都要设计不同的状态机；
- 从输出信号上分类，可分为Moore型状态机和Mealy型状态机。

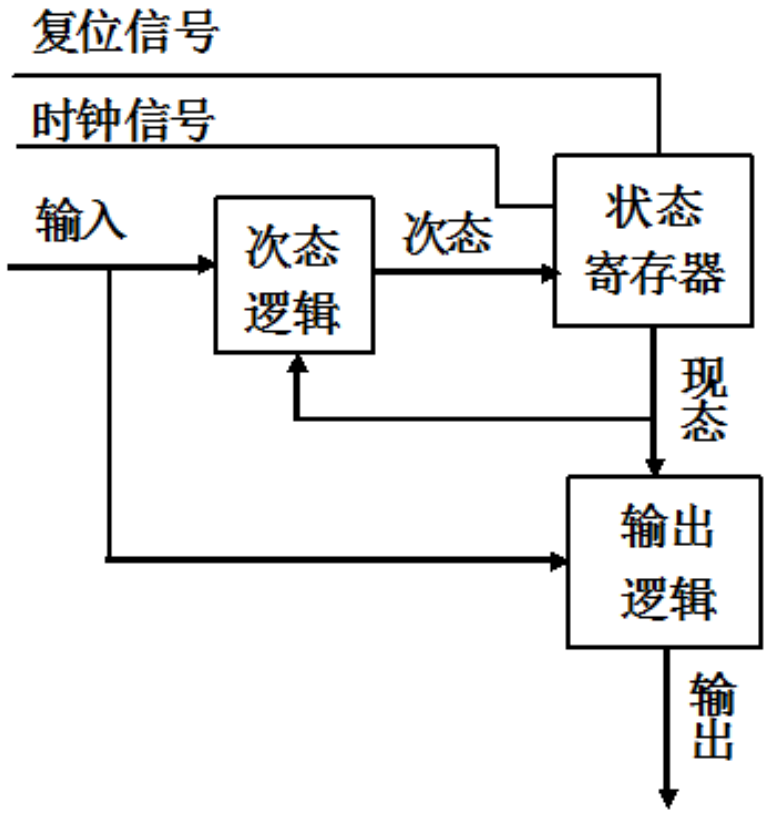
Moore型状态机

输出只与状态机当前的状态（现态）有关。



Mealy型状态机：

输出取决于状态机当前的状态（现态）和当前的输入。



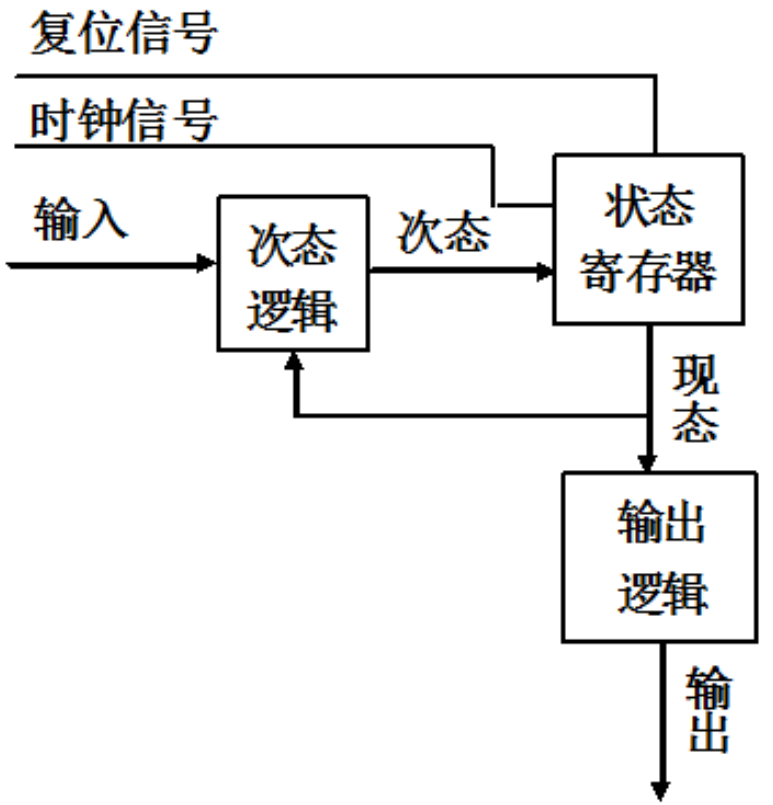
用Verilog描述有限状态机的基本原则

- 至少包括一个用于指定有限状态机状态的变量；
- 要有一个时钟用于状态同步；
- 要有状态转换条件和输出指定；
- 要有同步或异步复位信号。

状态机描述方式

状态机描述一般采用always语句和case语句：

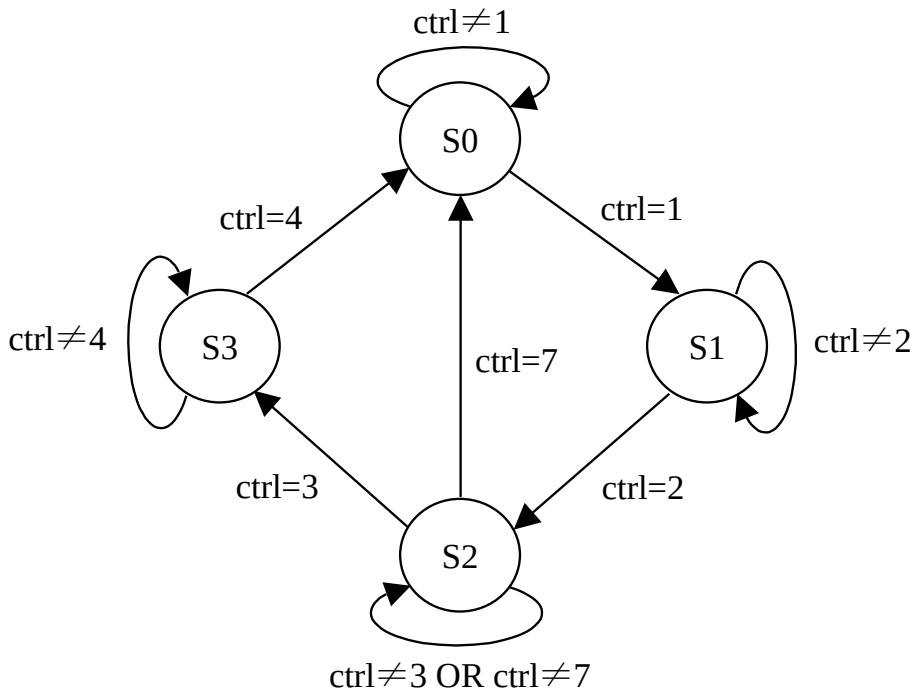
- 单always语句
- 双always语句
- 三always语句

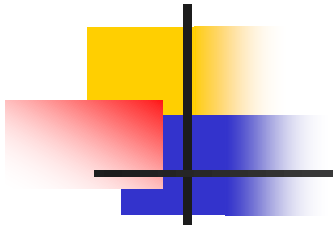


状态机的状态转换图如下，

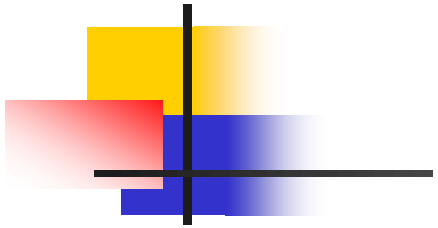
设计一个Moore型状态机，输出为：状态为S0时输出“00”，S1时输出“10”，S2时输出“11”，其他状态输出“01”；reset信号有效时，状态为S0。

设计一个Mealy型状态机，输出为：当状态为S2且输入为6时，输出为‘1’，其他情况输出为‘0’。 reset信号有效时，状态为S0。

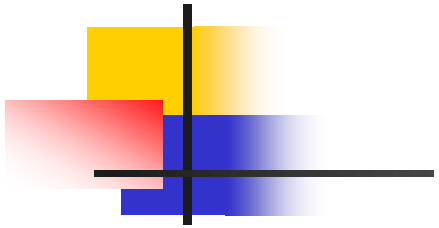




```
module FSM1 (clk, reset, ctrl, Moore, Mealy);
  input clk, reset;
  input [2:0] ctrl;
  output [1:0] Moore;
  output Mealy;
  reg [1:0] Moore;
  reg Mealy;
  parameter S0=0, S1=1, S2=2, S3=3; /*定义各种
                                    状态及其取值*/
  reg [1:0] st; //定义状态变量
```



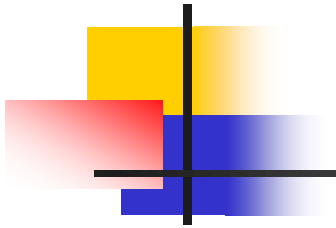
```
always @(posedge clk or negedge reset) begin
  if (!reset) st <= S0;
  else begin
    case (st)
      S0: begin
          if (ctrl==3'b001) st <= S1;
          else st <= S0; end
      S1: begin
          if (ctrl==3'b010) st <= S2;
          else st <= S1; end
      S2: begin
          if (ctrl==3'b011) st <= S3;
          else if (ctrl==3'b111) st <= S0;
          else st <= S2; end
      S3: begin
          if (ctrl==3'b100) st <= S0;
          else st <= S3; end
      default: st <= S0;
    endcase  end
  end
```



```
always @(st) begin  //Moore状态机输出
  case (st)
    S0: Moore <= 2'b00;
    S1: Moore <= 2'b10;
    S2: Moore <= 2'b11;
    S3: Moore <= 2'b01;
    Default: Moore <= 2'b00;
  endcase
end

always @(st) begin  //Mealy状态机输出
  if ((st==S2) & (ctrl==6))
    Mealy <= 1;
  else
    Mealy <= 0;
  end

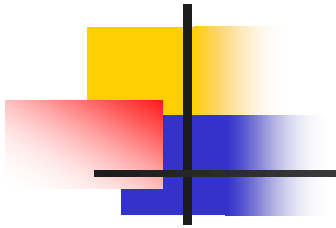
endmodule
```



- 状态机的各种状态取值用parameter来定义，要写出具体的值或编码，例如：

```
parameter [1:0] S0=0, S1=1, S2=2, S3=3;
reg [1:0] st;
```

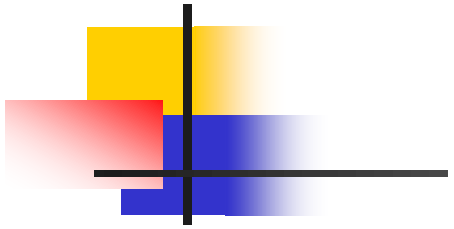
- 状态的取值也叫做状态编码，常用的有顺序码（二进制码）、格雷码（Gray）、一位热码（One-hot）等。此处采用顺序码，即用二进制00、01、10、11对状态进行编码；
- parameter后面的[1:0]可以省略掉，因为后面定义状态变量st的时候也说明了st的宽度。



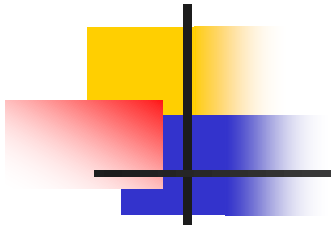
三always语句的Moore状态机

```
module FSM2 (clk, reset, ctrl, Moore);
  input clk, reset;
  input [2:0] ctrl;
  output [1:0] Moore;
  reg [1:0] Moore;
  parameter S0=0, S1=1, S2=2, S3=3;
  reg [1:0] current_state, next_state;

 always @(posedge clk or negedge reset) begin
   if (!reset) current_state <= S0;
   else current_state <= next_state;
 end
```



```
always @(current_state, ctrl) begin
    case (current_state)
      S0: begin
          if (ctrl==3'b001) next_state <= S1;
          else next_state <= S0; end
      S1: begin
          if (ctrl==3'b010) next_state <= S2;
          else next_state <= S1; end
      S2: begin
          if (ctrl==3'b011) next_state <= S3;
          else if (ctrl==3'b111) next_state <= S0;
          else next_state <= S2; end
      S3: begin
          if (ctrl==3'b100) next_state <= S0;
          else next_state <= S3; end
      default: next_state <= S0;
    endcase
  end
```



```
always @(current_state) begin
  case (current_state)
    S0: Moore <= 2'b00;
    S1: Moore <= 2'b10;
    S2: Moore <= 2'b11;
    S3: Moore <= 2'b01;
    Default: Moore <= 2'b00;
  endcase
end

endmodule
```